Projekt Spezifikation

Projektübersicht

Teammitglieder: Elisabeth Stagl, Valeriia Sineva
Arbeitstitel: Footprints

Die App ist ein mobiles Reisetagebuch, mit dem Nutzer ihre Reisen dokumentieren können. Auf der Startseite werden alle bereits erstellten Trips dargestellt. Innerhalb eines Trips können einzelne Stationen bzw. Steps angelegt werden, die ein Foto, eine persönliche Notiz, Datum und Standort enthalten.

Zusätzlich bietet die App einen Entdecken-Bereich, in dem Nutzer Länder bzw. Reiseziele vorgeschlagen bekommen und favorisieren können.

Die Zielgruppe sind Personen, die ihre Reiseerlebnisse an einem Ort festhalten möchten. Fotos, Notizen und Informationen zu besuchten Orten sind häufig auf unterschiedliche Apps verteilt. Die Anwendung soll diese Informationen in einem übersichtlichen digitalen Reisetagebuch zusammenführen und gleichzeitig Inspiration für zukünftige Reisen bieten.

Gewähltes Framework

Für die Entwicklung der App haben wir uns für **React Native (mit Expo Go)** entschieden, da wir bereits Erfahrung mit JavaScript und auch TypeScript sammeln konnten und React Native somit einfacher für uns zu erlernen ist.

React Native erfüllt gleichzeitig die technischen Anforderungen unseres Projekts. Die App benötigt unter anderem mehrere Screens mit Navigation, Formulare, Zugriff auf die Kamera und den GPS-Standort sowie die Verarbeitung externer API-Daten. Diese Funktionen lassen sich mit React Native und Expo umsetzen.

Als primäre Zielplattform wird Android verwendet. Die Anwendung wird dort entwickelt und getestet. Eine spätere Unterstützung von iOS wäre grundsätzlich möglich, ist im Rahmen dieses Projekts jedoch nicht vorgesehen, da uns aktuell keine macOS-/Xcode-Umgebung zur Verfügung steht.

Geplante Features (mit User Stories)

User Story 1 – Trips anzeigen → app/(tabs)/index.tsx, components/TripCard.tsx, store/tripStore.ts, services/database/tripStorage.ts

Als Nutzer möchte ich meine gespeicherten Trips auf der Startseite sehen, damit ich einen Überblick über meine bisherigen Reisen habe.

Akzeptanzkriterium: Die Funktion ist fertig, wenn alle lokal gespeicherten Trips als Liste angezeigt werden und ein Trip ausgewählt werden kann.

User Story 2 – Trips verwalten → app/trip/create.tsx, app/trip/edit.tsx, app/(tabs)/index.tsx, components/TripForm.tsx, store/tripStore.ts, services/database/tripStorage.ts

Als Nutzer möchte ich Trips erstellen, bearbeiten und löschen können, damit ich meine Reisen verwalten kann.

Akzeptanzkriterium: Trips können mit den benötigten Informationen erstellt, nachträglich bearbeitet und gelöscht werden. Änderungen werden lokal gespeichert.

User Story 3 – Steps anzeigen → app/trip/[id].tsx, components/StepCard.tsx, store/tripStore.ts, services/database/tripStorage.ts

Als Nutzer möchte ich einen Trip öffnen und die dazugehörigen Steps sehen, damit ich meine Reise nachvollziehen kann.

Akzeptanzkriterium: Nach Auswahl eines Trips werden die Trip-Informationen und alle zugehörigen Steps angezeigt.

User Story 4 – Aktuellen Standort erfassen → app/step/create.tsx, app/step/edit.tsx, components/StepForm.tsx, services/nativeFeatures.ts, store/tripStore.ts, services/database/tripStorage.ts

Als Nutzer möchte ich meinen aktuellen Standort zu einem Step hinzufügen, damit gespeichert wird, wo die Erinnerung entstanden ist.

Akzeptanzkriterium: Die Funktion ist fertig, wenn die App die notwendige Standortberechtigung anfragt, die GPS-Koordinaten des Geräts ermittelt und diese mit dem Step speichert.

User Story 5 – Foto hinzufügen → app/step/create.tsx, app/step/edit.tsx, components/StepForm.tsx, services/nativeFeatures.ts, store/tripStore.ts, services/database/tripStorage.ts

Als Nutzer möchte ich beim Erstellen eines Steps ein Foto aufnehmen, damit ich eine visuelle Erinnerung zu diesem Ort speichern kann.

Akzeptanzkriterium: Die Funktion ist fertig, wenn die App die Kameraberechtigung anfragt, die Kamera geöffnet werden kann und das aufgenommene Foto dem Step zugeordnet wird.

User Story 6 – Notiz hinzufügen → app/step/create.tsx, app/step/edit.tsx, app/step/[id].tsx, components/StepForm.tsx, store/tripStore.ts, services/database/tripStorage.ts

Als Nutzer möchte ich einem Step eine persönliche Notiz hinzufügen, damit ich festhalten kann, was ich an diesem Ort erlebt habe.

Akzeptanzkriterium: Die Funktion ist fertig, wenn eine Beschreibung eingegeben, gespeichert und beim jeweiligen Step angezeigt werden kann.

User Story 7 – Reiseziele entdecken → app/(tabs)/explore.tsx, components/CountryCard.tsx, services/countriesService.ts, store/useCountriesStore.ts, services/database/countriesStorage.ts, services/database/dailyCountriesStorage.ts

Als Nutzer möchte ich im Entdecken-Bereich mögliche Reiseziele durchsuchen, damit ich Inspiration für zukünftige Reisen erhalten kann.

Akzeptanzkriterium: Die Funktion ist fertig, wenn Daten zu Ländern bzw. Reisezielen über eine externe API (REST Countries API) geladen und in der App angezeigt werden.

User Story 8 – Reiseziel als Favorit markieren → components/CountryCard.tsx, app/(tabs)/favourites.tsx, store/useCountriesStore.ts, services/database/favouritesStorage.ts

Als Nutzer möchte ich ein vorgeschlagenes Reiseziel zu meiner Favoritenliste hinzufügen, damit ich das Reiseziel für später speichern kann.

Akzeptanzkriterium: Die Funktion ist fertig, wenn beim Klicken auf den Herz-Button das ausgewählte Reiseziel in der Favoritenliste gespeichert wird und dort später wieder aufgerufen werden kann.

User Story 9 – Trips offline verwenden → store/tripStore.ts, store/useCountriesStore.ts, services/database/database.ts, services/database/tripStorage.ts, services/database/countriesStorage.ts, services/database/dailyCountriesStorage.ts, services/database/favouritesStorage.ts

Als Nutzer möchte ich bereits gespeicherte Trips und zuvor geladene Reiseziele, sowie Favoriten auch ohne Internetverbindung ansehen können, damit die wichtigsten Inhalte der App unterwegs verfügbar bleiben.

Akzeptanzkriterium: Die Funktion ist fertig, wenn lokale Trips und Steps sowie zuletzt erfolgreich geladene API-Daten ohne aktive Internetverbindung angezeigt werden können.

Screen / Navigation Structure

Die App verwendet Expo Router und kombiniert eine Tab-Navigation für die Hauptbereiche (Home, Entdecken und Favoriten) mit Stack-Navigation für Detail- und Erstellungsseiten.

1. Startseite (Home Screen)

Der Home-Screen ist der Startbildschirm der Anwendung und zeigt alle gespeicherten Trips als Liste. Ein Trip kann ausgewählt werden, um dessen Detailansicht zu öffnen.

State: Liste der gespeicherten Trips

2. Trip erstellen (Create Trip Screen)

Durch Klicken auf den +-Button gelangt man zum Trip-erstellen-Screen. Hier kann ein neuer Trip angelegt werden. Dazu werden grundlegende Informationen wie Name, Start- und Enddatum eingegeben.

State: Formulardaten des neuen Trips

3. Trip Details (Details Screen)

Die Detailansicht zeigt die Informationen eines ausgewählten Trips sowie alle dazugehörigen Steps. Von hier aus kann, durch Klick auf den +-Button, ein neuer Step erstellt werden.

State: Ausgewählter Trip und zugehörige Steps

4. Step erstellen (Create Step)

Auf diesem Screen kann ein neuer Step innerhalb eines Trips erstellt werden. Der Nutzer kann eine Notiz eingeben, den aktuellen GPS-Standort erfassen und ein Foto aufnehmen.

State: Beschreibung, Datum/Uhrzeit, GPS-Koordinaten, Foto sowie Berechtigungsstatus

5. Entdecken Screen (Discover Screen)

Der Entdecken-Screen lädt Reiseziele bzw. Länder über eine externe API. Nutzer können die verfügbaren Ziele durchsuchen. Durch Auswahl eines Ergebnisses können weitere Informationen zum jeweiligen Reiseziel angezeigt werden.

State: API-Daten, Lade- und Fehlerstatus

6. Favourites Screen

Der Favourites-Screen zeigt alle vom Nutzer gespeicherten Reiseziele als Liste. Ein Reiseziel kann über den Herz-Button wieder aus der Favoritenliste entfernt werden.

State: Liste der gespeicherten Favoriten.

Navigation Flow

Die Hauptbereiche sind über die Tab-Navigation erreichbar:
Home | Entdecken | Favoriten

Das Hinzufügen neuer Inhalte erfolgt über einen +-Button. Auf der Startseite führt dieser zum Erstellen eines neuen Trips. Innerhalb eines Trips kann über einen +-Button ein neuer Step hinzugefügt werden.

- Home → Trip Details (Stack)
- Home → +-Button → Create Trip (Stack)
- Trip Details → +-Button → Create Step (Stack)

API and Data Sources

Für den Entdecken-Bereich verwenden wir die REST Countries API. Über die API können Länder gesucht und Informationen wie Name, Flagge, Hauptstadt, Region und Sprachen abgerufen werden.

Die Länder werden auf dem Entdecken-Screen als Cards dargestellt. Für reguläre API-Anfragen wird ein API-Key benötigt. Erfolgreich geladene Daten werden zusätzlich lokal zwischengespeichert, sodass bei fehlender Internetverbindung der zuletzt gespeicherte Stand angezeigt werden kann.

Native Feature Usage

Die App verwendet GPS und die Kamera als native Gerätefunktionen. Beim Erstellen eines Steps kann der aktuelle Standort erfasst und gemeinsam mit dem Step gespeichert werden. Über die Kamera kann zusätzlich ein Foto aufgenommen und dem Step zugeordnet werden. Diese Funktionen ermöglichen es, Reiseerinnerungen direkt mit dem jeweiligen Ort und einem Foto zu dokumentieren.

Offline / Persistence Strategy

Trips, Steps, Notizen, GPS-Koordinaten und Referenzen auf Fotos werden lokal auf dem Gerät gespeichert. Dafür soll SQLite verwendet werden, während Fotos im lokalen Dateisystem gespeichert werden.

Auch erfolgreich geladene Daten aus der externen API werden lokal zwischengespeichert, also gecacht. Dadurch kann bei fehlender Internetverbindung weiterhin der zuletzt gespeicherte Stand angezeigt werden. Sobald wieder eine Internetverbindung verfügbar ist, können die API-Daten aktualisiert werden.

Technical Approach & Architecture

Für die Navigation verwenden wir Expo Router mit File-Based Routing. Die App wird in Routes/Screens, wiederverwendbare Components, Hooks, Services und State Management aufgeteilt.

```
footprints/
├── app/                    # File-based routes
│   ├── _layout.tsx         # Root layout & providers
│   │
│   ├── (tabs)/             # Tab navigation group
│   │   ├── _layout.tsx     # Bottom tabs definition
│   │   ├── index.tsx       # Home / Trips
│   │   └── discover.tsx    # Entdecken
│   │   └── favourites.tsx  # Favoriten
│   │
│   ├── trip/
│   │   ├── trip_details.tsx  # Trip details
│   │   └── create.tsx        # Create trip
│   │
│   └── step/
```

```
│   └── create.tsx          # Create step
│
└── src/
    ├── components/         # Reusable UI components
    ├── services/           # API & storage handlers, SQLite, GPS, Kamera
    └── store/              # State management (Zustand)
```

Changelog

Neue Projektstruktur:

```
CROSS_App/
├── README.md
├── .github/
│   └── workflows/
│       └── ci.yml
├── documentation/
│   ├── project-requirements.pdf
│   ├── project-specification.pdf
│   └── screenshots/
└── footprints/
    ├── app.json
    ├── package.json
    ├── eslint.config.js
    ├── jest.config.js
    ├── tests/
    └── src/
        ├── app/
        ├── components/
        ├── constants/
        ├── hooks/
        ├── services/
        │   └── database/
        ├── store/
        ├── types/
        └── utils/
```

Änderungen:

1. **API- und Speicherstrategie**
   Der Länderkatalog wird beim ersten Laden heruntergeladen und dauerhaft in SQLite gespeichert. Weitere API-Anfragen sind nur erforderlich, wenn kein gespeicherter Katalog vorhanden ist. Dadurch werden unnötige API-Aufrufe, Datenverbrauch und Ladezeiten reduziert.
2. **Offline-Empfehlungen**
   Statt offline ausschließlich die letzte Auswahl anzuzeigen, werden die täglichen Empfehlungen aus dem lokal gespeicherten Länderkatalog zusammengestellt. Auch „Discover more“ verwendet diese Daten. Dadurch können Nutzer ohne Internetverbindung andere Reiseziele entdecken.

Zusätzliche Features:

1. **Trip-Zeitraum anpassen**
   Liegt das Datum eines Steps außerhalb des bisherigen Reisezeitraums, fragt die App, ob dieser erweitert werden soll.
2. **Ortsnamen anzeigen**
   Gespeicherte GPS-Koordinaten werden nach Möglichkeit in einen lesbaren Ortsnamen umgewandelt.
3. **Dark mode**
   Die Oberfläche passt sich automatisch an das helle oder dunkle Systemdesign des Geräts an.

Appendix

